

HDOC Day - 16

Name- Aaryan Pratap Singh

Sap id- 590032842

Batch- 15

1. Algorithm

Main Program

1. Start.
2. Declare `choice`.
3. Display the menu:
 - 1 — Check increasing order.
 - 2 — Check decreasing order.
 - 3 — Find largest and smallest digit.
 - 4 — Find second largest and second smallest digit.
4. Read `choice`.
5. Use `switch(choice)`.
6. For choice 1, call `increasingOrder()`.
7. For choice 2, call `decreasingOrder()`.
8. For choice 3, call `largestSmallest()`.
9. For choice 4, call `secondLargestSmallest()`.
10. For any other choice, display "Invalid choice".
11. Stop.

Function 1: Increasing Order

1. Read a positive integer.
2. Find the divisor corresponding to its leftmost digit.
3. Extract the digits from left to right.
4. Compare every digit with the next digit.
5. If `previous >= current`, the digits are not strictly increasing.
6. Otherwise, they are in increasing order.
7. Display the result.

Function 2: Decreasing Order

1. Read a positive integer.
2. Find the divisor corresponding to its leftmost digit.
3. Extract the digits from left to right.
4. Compare every digit with the next digit.
5. If `previous <= current`, the digits are not strictly decreasing.
6. Otherwise, they are in decreasing order.
7. Display the result.

Function 3: Largest and Smallest Digit

1. Read a positive integer.
2. Initialize `largest = -1` and `smallest = 10`.
3. Extract each digit using `% 10`.
4. If the digit is greater than `largest`, update `largest`.
5. If the digit is smaller than `smallest`, update `smallest`.
6. Remove the digit using `/ 10`.
7. Repeat until all digits are processed.
8. Display the largest and smallest digits.

Function 4: Second Largest and Second Smallest Digit

1. Read a positive integer.
2. Initialize variables for largest, second largest, smallest, and second smallest.
3. Extract each digit.
4. Update the largest and second-largest **distinct** digits.
5. Update the smallest and second-smallest **distinct** digits.
6. Continue until all digits are processed.
7. If second largest/second smallest do not exist, display an appropriate message.
8. Otherwise, display both results.

2. Test Case Table

Test Case	Choice	Input Number	Expected Output	Actual Output	Result
1	1	12359	Digits are in increasing order	Digits are in increasing	Pass
2	1	1224	Digits are not in increasing order	Digits are not in increasing	Pass
3	2	97520	Digits are in decreasing order	Digits are in decreasing	Pass
4	2	97570	Digits are not in decreasing order	Digits are not in	Pass
5	3	50824	Largest = 8, Smallest = 0	Largest = 8, Smallest = 0	Pass
6	4	58320	Second largest = 5, Second smallest = 2	Second largest = 5, Second smallest = 2	Pass
7	4	7777	Second largest and second smallest distinct digits do not exist	Same	Pass

Here, “second largest” and “second smallest” mean **second largest/smallest distinct digits**.

3. C Program

```
#include <stdio.h>

/* Function declarations */
void increasingOrder(void);
```

```

void decreasingOrder(void);
void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;

    printf("==== DIGIT OPERATIONS MENU =====\n");
    printf("1. Check increasing order\n");
    printf("2. Check decreasing order\n");
    printf("3. Find largest and smallest digit\n");
    printf("4. Find second largest and second smallest digit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            increasingOrder();
            break;

        case 2:
            decreasingOrder();
            break;

        case 3:
            largestSmallest();
            break;

        case 4:
            secondLargestSmallest();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}

/* Function to check increasing order */
void increasingOrder(void)

```

```

{
    long long n, temp, divisor = 1;
    int prev, curr, increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0)
    {
        printf("Invalid input\n");
        return;
    }

    temp = n;

    /* Find divisor for leftmost digit */
    while (temp >= 10)
    {
        divisor = divisor * 10;
        temp = temp / 10;
    }

    prev = n / divisor;
    n = n % divisor;
    divisor = divisor / 10;

    while (divisor > 0)
    {
        curr = n / divisor;

        if (prev >= curr)
        {
            increasing = 0;
            break;
        }

        prev = curr;
        n = n % divisor;
        divisor = divisor / 10;
    }

    if (increasing)
        printf("Digits are in increasing order\n");
    else
        printf("Digits are not in increasing order\n");
}

```

```

}

/* Function to check decreasing order */
void decreasingOrder(void)
{
    long long n, temp, divisor = 1;
    int prev, curr, decreasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0)
    {
        printf("Invalid input\n");
        return;
    }

    temp = n;

    /* Find divisor for leftmost digit */
    while (temp >= 10)
    {
        divisor = divisor * 10;
        temp = temp / 10;
    }

    prev = n / divisor;
    n = n % divisor;
    divisor = divisor / 10;

    while (divisor > 0)
    {
        curr = n / divisor;

        if (prev <= curr)
        {
            decreasing = 0;
            break;
        }

        prev = curr;
        n = n % divisor;
        divisor = divisor / 10;
    }
}

```

```

    if (decreasing)
        printf("Digits are in decreasing order\n");
    else
        printf("Digits are not in decreasing order\n");
}

/* Function to find largest and smallest digit */
void largestSmallest(void)
{
    long long n;
    int digit;
    int largest = -1;
    int smallest = 10;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0)
    {
        printf("Invalid input\n");
        return;
    }

    while (n > 0)
    {
        digit = n % 10;

        if (digit > largest)
            largest = digit;

        if (digit < smallest)
            smallest = digit;

        n = n / 10;
    }

    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

/* Function to find second largest and second smallest digit
*/
void secondLargestSmallest(void)
{
    long long n;

```

```

int digit;

int largest = -1;
int secondLargest = -1;

int smallest = 10;
int secondSmallest = 10;

printf("Enter a positive integer: ");
scanf("%lld", &n);

if (n <= 0)
{
    printf("Invalid input\n");
    return;
}

while (n > 0)
{
    digit = n % 10;

    /* Find largest and second largest */
    if (digit > largest)
    {
        secondLargest = largest;
        largest = digit;
    }
    else if (digit < largest && digit > secondLargest)
    {
        secondLargest = digit;
    }

    /* Find smallest and second smallest */
    if (digit < smallest)
    {
        secondSmallest = smallest;
        smallest = digit;
    }
    else if (digit > smallest && digit < secondSmallest)
    {
        secondSmallest = digit;
    }

    n = n / 10;
}

```

```

if (secondLargest == -1 || secondSmallest == 10)
{
    printf("Second largest and second smallest "
           "distinct digits do not exist\n");
}
else
{
    printf("Second largest digit = %d\n", secondLargest);
    printf("Second smallest digit = %d\n",
secondSmallest);
}
}

```

4. Execution

Case 1

```

[aaryanrajput@aaryans-macbook ~ % nano day16.c
[aaryanrajput@aaryans-macbook ~ % clang day16.c -o day16.c
[aaryanrajput@aaryans-macbook ~ % ./day16.c
===== DIGIT OPERATIONS MENU =====
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 1
Enter a positive integer: 12359
Digits are in increasing order
aaryanrajput@aaryans-macbook ~ % █

```

Case 2

```

[aaryanrajput@aaryans-macbook ~ % ./day16.c
===== DIGIT OPERATIONS MENU =====
1. Check increder
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 2
Enter a positive integer: 97520
Digits are in decreasing order
aaryanrajput@aaryans-macbook ~ % █

```


Case 3

```
[aaryanrajput@aaryans-macbook ~ % ./day16.c
===== DIGIT OPERATIONS MENU =====
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 3
Enter a positive integer: 50824
Largest digit = 8
Smallest digit = 0
aaryanrajput@aaryans-macbook ~ %
```

Case 4

```
[aaryanrajput@aaryans-macbook ~ % ./day16.c
===== DIGIT OPERATIONS MENU =====
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice: 4
Enter a positive integer: 58320
Second largest digit = 5
Second smallest digit = 2
aaryanrajput@aaryans-macbook ~ %
```